

Joint Compute-Control Budgeting for Receding-Horizon Diffusion Policy

Iso-Compute Allocation of Denoising and Replanning for Efficient Robotic Manipulation

Abstract

Diffusion Policy uses iterative action denoising and receding-horizon execution. This project reproduces the official low-dimensional Push-T checkpoint with the upstream workspace and runner, obtaining a 50-seed mean score of 0.919 in the current compatibility environment. It then evaluates how to allocate a fixed inference budget between denoising quality and replanning frequency in a deterministic Push-T-style surrogate. The optimized version uses 20 surrogate seeds, two scheduler variants, and a supplementary official fixed-(k,h) frontier.

Problem

We define k as denoising steps per policy call, h as executed controls before replanning, and $B \sim k/h$ as amortized compute per control step. The grid evaluates k in $\{2,4,8,16\}$ and h in $\{1,2,4,8\}$, with main analysis on $B=1$, $B=2$, and $B=4$ frontiers.

Official Reproduction

The official source was acquired through a GitHub codeload archive after git clone instability, and the 1044185793-byte Push-T checkpoint was downloaded from the project server. The 50-seed official runner result is 0.919; the checkpoint filename reports 0.969, so the difference is recorded as an environment-version reproduction gap.

Official Fixed Frontier

With the official checkpoint, a 20-seed fixed-(k,h) high-budget frontier strongly favors denoising depth: the best tested point is ($k=100$, $h=8$) with score 0.949 and success 0.950. This official check constrains the surrogate claim and shows that allocation behavior is model-dependent.

LeRobot Supplement

The LeRobot lerobot/diffusion_pusht model card reports 500-episode average max reward 0.955 and success 65.4%. The local mirror-downloaded checkpoint loads and runs in gym-pusht; a 300-step, 100-denoising-step smoke test reached max reward 0.627. Because the file SHA did not match HF LFS metadata, local rollout results are marked as smoke verification.

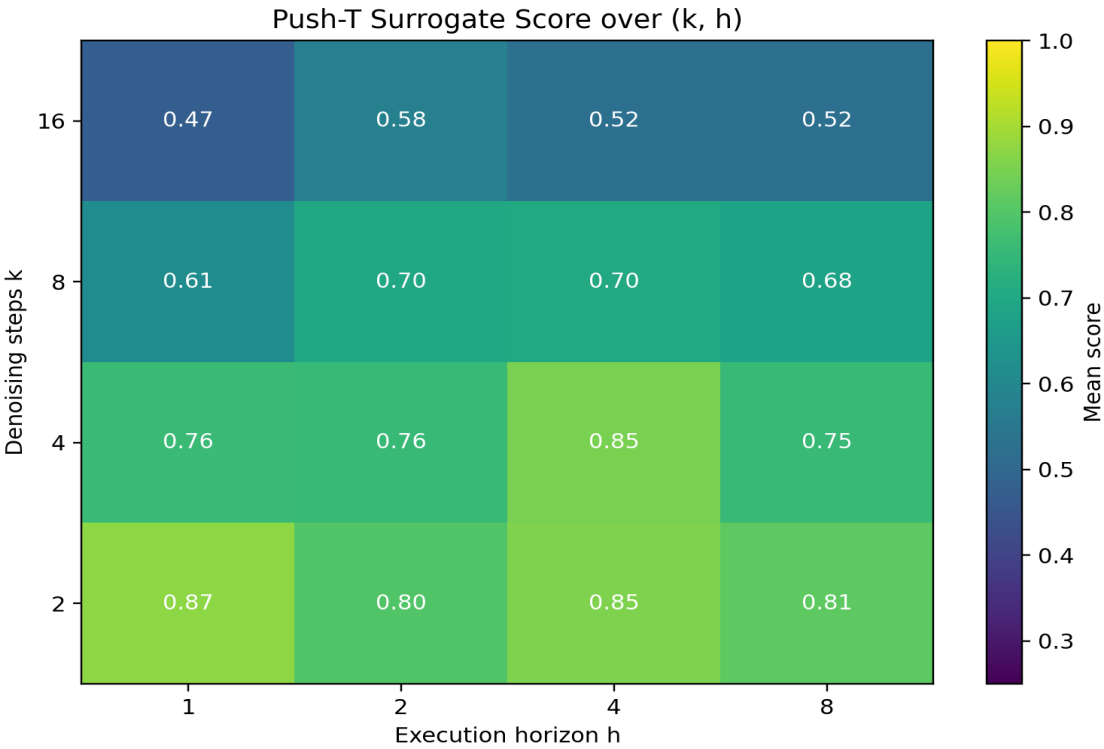
Method

A fixed runner evaluates every (k,h). A training-free score-oriented joint scheduler operates on the $B=2$ frontier. It selects (4,2) in free-space approach and final alignment, and (2,1) during close-contact pushing or unstable progress. A safe variant switches to (2,1) earlier to favor reliability.

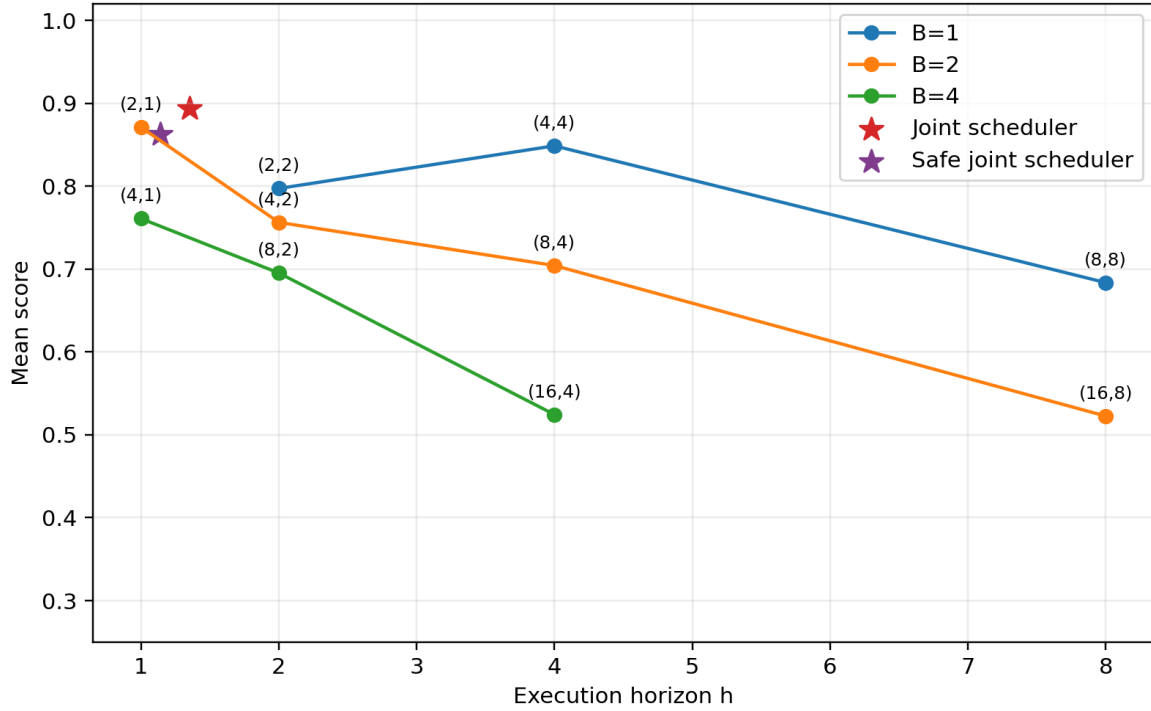
Main Comparison

Method	B	Score	Success	Calls	NFE	Smoothness
Default DP-style (8,4)	2.00	0.704	0.550	30.0	240.0	0.035
Best fixed B=2 (2,1)	2.00	0.871	0.900	120.0	240.0	0.420

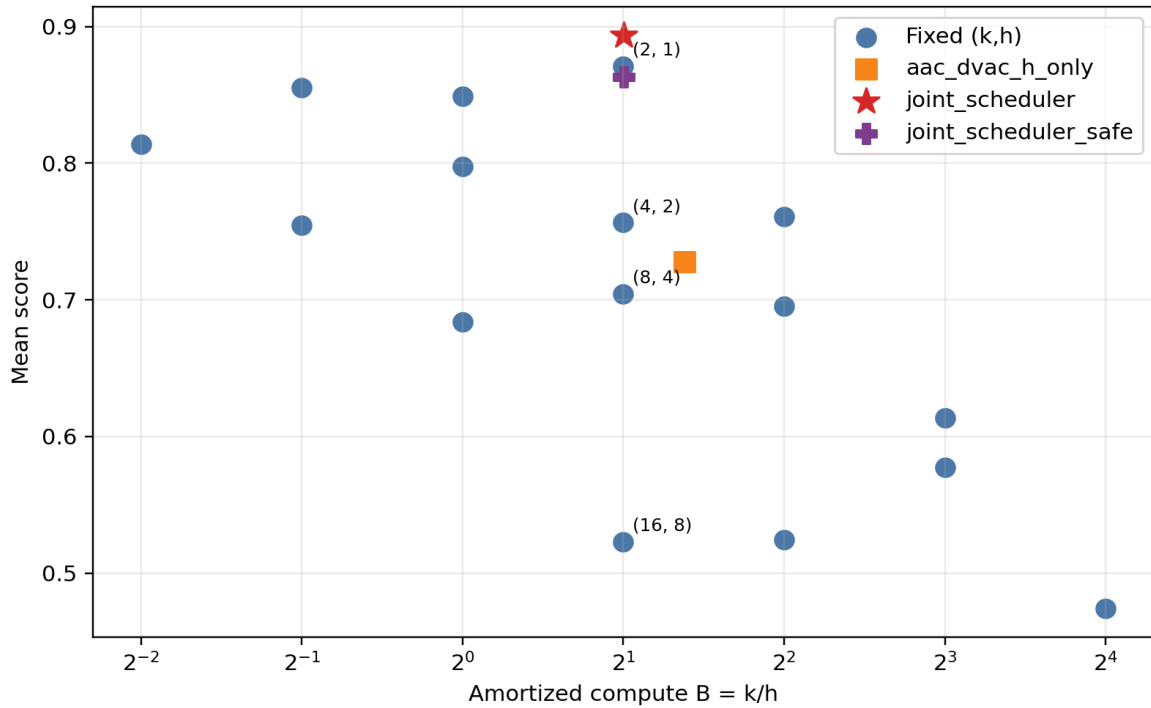
Method	B	Score	Success	Calls	NFE	Smoothness
Denoising-only point (4,2)	2.00	0.756	0.700	60.0	240.0	0.105
Long denoise/chunk (16,8)	2.00	0.523	0.200	15.0	240.0	0.009
AAC/DVAC-style h-only	2.60	0.728	0.550	39.0	312.4	0.046
Joint scheduler (score)	2.00	0.893	0.850	89.2	240.6	0.257
Joint scheduler (safe)	2.00	0.863	0.950	105.3	240.3	0.340



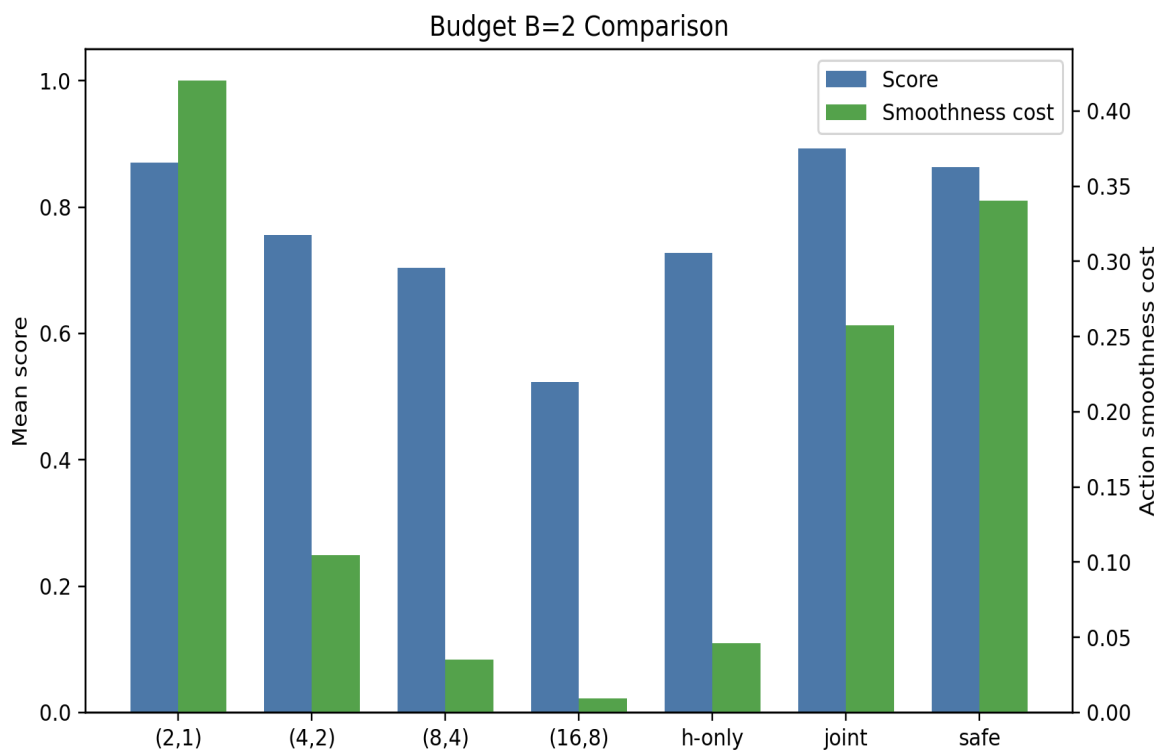
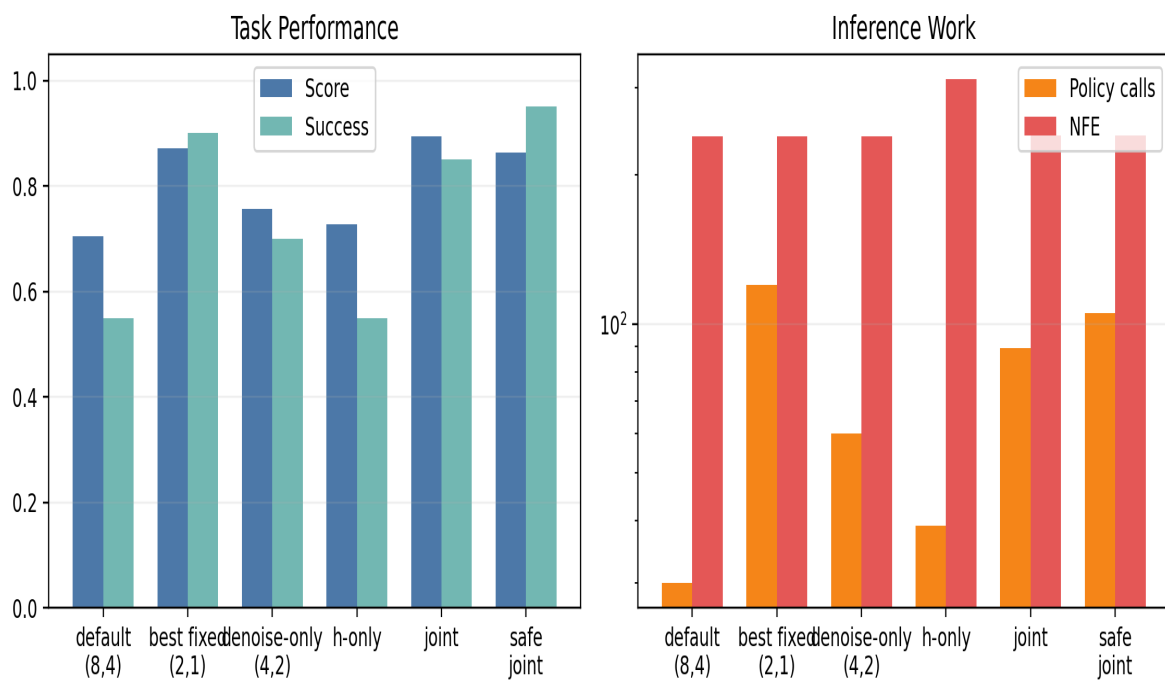
Iso-Compute Frontiers: Denoising vs Replanning

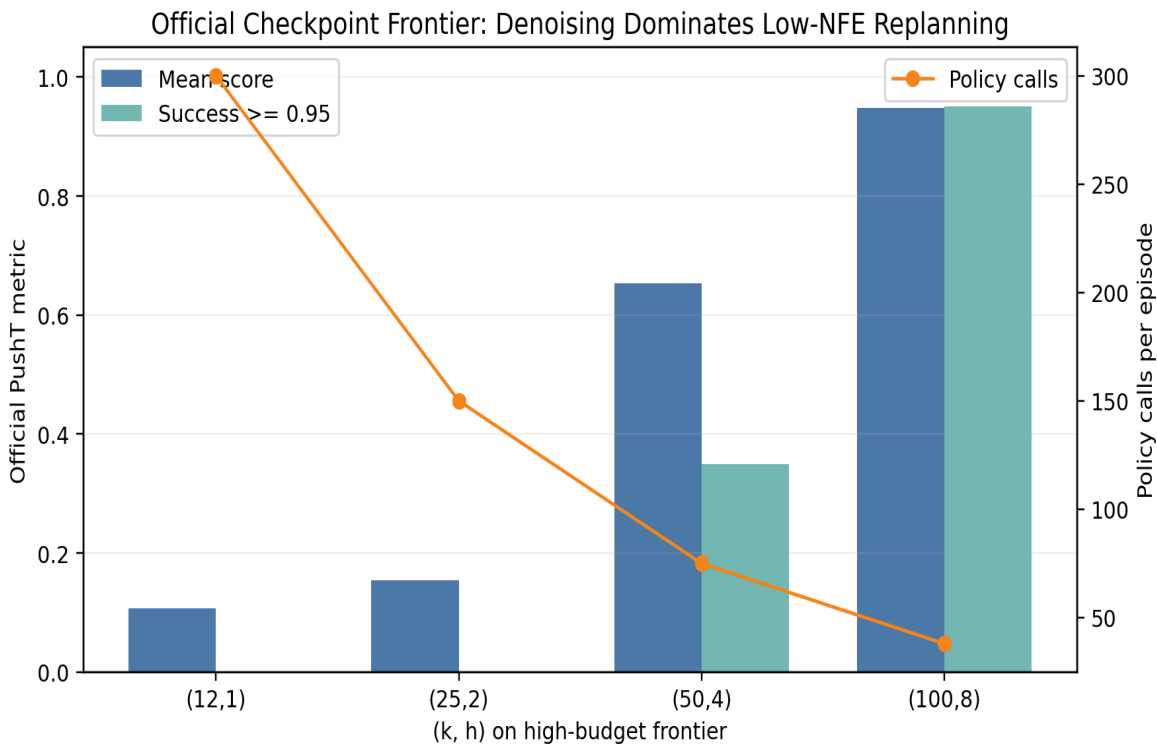
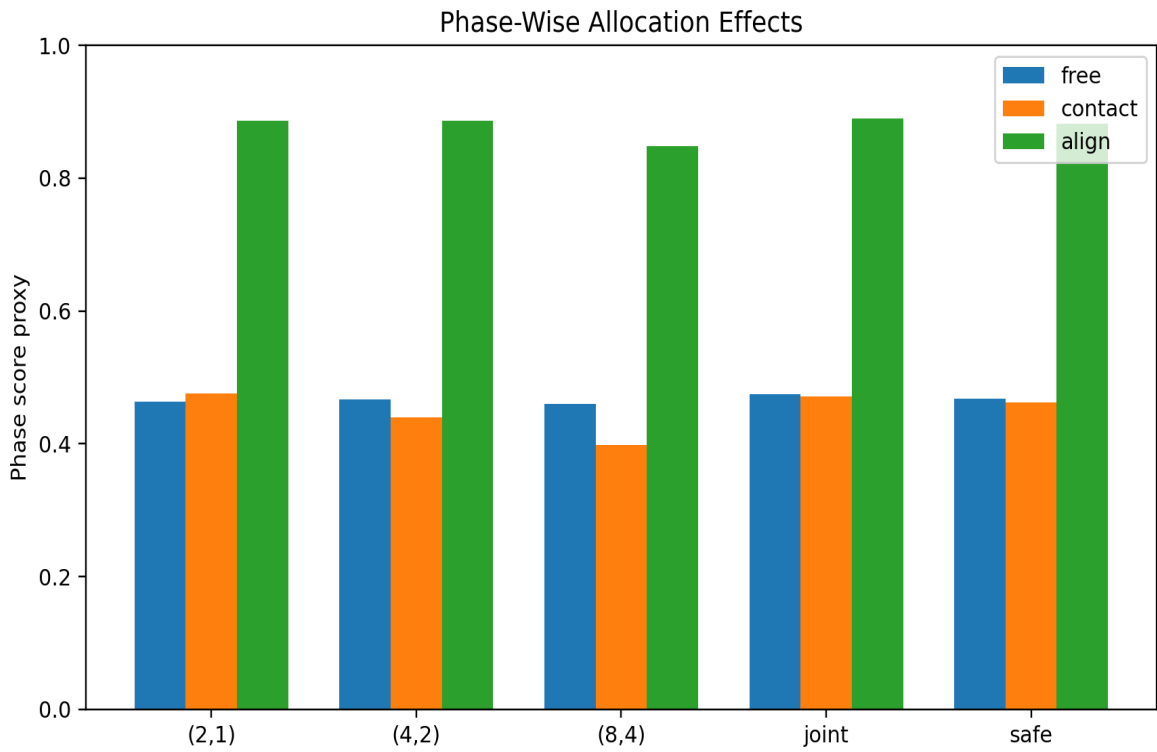


Score vs Amortized Inference Budget



Main Baseline Comparison under $B \approx 2$





Results

The score-oriented scheduler reaches score 0.893 at average $B=2.005$, compared with 0.871 for the best fixed score baseline (2,1). It reduces policy calls from 120.0 to 89.2 and smoothness cost from 0.420 to 0.257. The safe scheduler reaches success 0.950 and policy calls 105.3.

Limitations

The scheduler-improvement numeric results are from the surrogate, not official Diffusion Policy checkpoint rollouts. The official checkpoint run uses compatibility patches for Python 3.12 / Gym 0.26, and the LeRobot mirror checkpoint is executable locally but not SHA-verified. The official fixed-(k,h) frontier shows model-dependent allocation behavior. The compute model is approximate, the scheduler is rule-based, and the experiment is limited to one task family.

Conclusion

The experiment supports the hypothesis that denoising and replanning should be allocated jointly. Contact phases favor frequent replanning; easier phases can use a longer chunk and more denoising while staying on the same compute frontier.

References

Diffusion Policy project page: <https://diffusion-policy.cs.columbia.edu/>. Official code: https://github.com/real-stanford/diffusion_policy. LeRobot model card: https://huggingface.co/lerobot/diffusion_pusht. Related work: ACT / arXiv 2304.13705, Consistency Policy / arXiv 2405.07503, DDIM / arXiv 2010.02502, DPM-Solver / arXiv 2206.00927, and Unpacking the Individual Components of Diffusion Policy / arXiv 2412.00084.